Ciência da **Computação**

Prof. Luciano Rodrigo Ferretto

Um breve parênteses...

Compile Time vs Run Time

COMPILE TIME

Errors detected at compilation

```
int c = "hello";
```

Compile-time error: String cannot be assighed to int

RUN TIME

Errors detected during execution

```
int a = 10;
int b = 0;
int result = a / b;
```

Run-time error: Division by zero

Estruturas Condicionais – Instrução **if**

- Olá, turma! Já que vocês estão familiarizados com a estrutura condicional **if** em Python, será mais fácil compreender a mesma ideia em Java.

- A estrutura **if** é usada para executar um bloco de código **somente se uma condição especificada for verdadeira**. Vamos explorar como isso funciona em Java e como podemos utilizar suas diversas formas e derivações.

Relembrando em Python **if**

1st Condition is True

```
let number = 5
if number > 0 :
    # code

elif number < 0 :
    # code

else :
    # code


# code after if
```

2nd Condition is True

```
let number = -5
if number > 0 :
    # code

elif number < 0 :
    # code

else :
    # code


# code after if
```

All Conditions are False

```
let number = 0
if number > 0 :
    # code

elif number < 0 :
    # code

else :
    # code


# code after if
```


Ciência da Computação

Sintaxe Básica da Instrução **if** em Java

- Assim como no Python, no Java podemos controlar a execução de uma instrução, ou de uma sequência de instruções (bloco), a partir de uma condição específica.
- Quando estamos tratando uma sequência de instruções, estas devem estar entre **chaves {}**.
- Quando é uma linha de instrução somente, não há a necessidade das **chaves {}**, porém podem ser utilizadas também.

```
if(condição)
{
  sequência de instruções
}
else
{
  sequência de instruções
}
```

```
if(condição) instrução;
else instrução;
```

Exemplo da Instrução **if** em Java

- Vamos imaginar que queremos construir um algoritmo que receba duas notas de um aluno, e apresente o resultado conforme a média aritmética das notas.

```java
import java.util.Scanner; //Biblioteca utilizada para acessar os dados digitados pelo usuário.
class CalculaMedia { //Nessa classe não vamos realizar o tratamento de exceções.

    public static void main(String args[]){
        Scanner lerTeclado = new Scanner(System.in); //Inicializa o objeto Scanner passando qual será a
                                                     // origem dos dados. Neste caso será a entrada no terminal
        //Solicita ao usuário que informe suas
        System.out.println("Informe o valor da primeira Nota:");
        double nota01 = lerTeclado.nextDouble();
        System.out.println("Informe o valor da segunda Nota:");
        double nota02 = lerTeclado.nextDouble();
        lerTeclado.close();
        //Calcula a média aritmética
        double mediaAritimetica = (nota01 + nota02) / 2;
        System.out.printf("A média aritimética é %.2f \n %n", mediaAritimetica);

        if (mediaAritimetica ≥ 7) { //exemplo utilizando com chaves {}
            System.out.println("PARABÉNS!!! Você foi aprovado");
        } else                      //como é apenas uma instrução, também pode ser utiizada sem as chaves {}
            System.out.println("Infelizmente você não alcançou a média. :-(");
    }
}
```


Ciência da Computação

```java
//Exemplo utilizando a estrutura if comum
if (mediaAritimetica ≥ 7) { // com o uso chaves {}
    System.out.println("PARABÉNS!!! Você foi aprovado");
} else                    // sem o uso das chaves {}
    System.out.println("Infelizmente você não alcançou a média :-(");
```

Instrução **if** aninhado

Um ***if** aninhado* é uma instrução **if** que é alvo de outro **if** ou **else**. Os **if**s aninhados são muito comuns em programação. O importante a lembrar sobre **if**s aninhados em Java é o fato de que uma instrução **else** será sempre referente à instrução **if** mais próxima que estiver dentro do mesmo bloco e ainda não estiver associada a um **else**. Aqui está um exemplo:

```
if(i == 10) {
  if(j < 20) a = b;
  if(k > 100) c = d;
  else a = c; // esse else é referente a if(k > 100)
}
else a = d; // esse else é referente a if(i == 10)
```

```java
        if (mediaAritimetica ≥ 7)
            System.out.println("PARABÉNS!!! Você foi aprovado");
        else {
            if (mediaAritimetica ≥ 3) {
                System.out.println("Infelizmente você deverá realizar o Exame :-(");
            } else
                System.out.println("Infelizmente você foi Reprovado! :(");
        }
```

Instrução **if** aninhado – Escada if-else-if

- Um método bastante utilizado de If aninhado em programação é a *escada* **if-else-if** (o "*elif*" do python)

```
if (condição)
        instrução;
else if (condição) {
        sequencia_instruções;
} else if (condição)
        instrução;
else
        instrução;
```

```
18        if (mediaAritimetica ≥ 7)
19            System.out.println("PARABÉNS!!! Você foi aprovado");
20        else if (mediaAritimetica ≥ 3)
21            System.out.println("Infelizmente você deverá realizar o Exame :-(");
22        else
23            System.out.println("Infelizmente você foi Reprovado! :(");
24
```

Atividade Prática: Programando um Radar de Trânsito

- **Objetivo:**
  Crie um programa em Java que receba a **velocidade máxima permitida** em uma via e a **velocidade registrada** de um veículo. O programa deve exibir a situação do condutor seguindo as regras abaixo:

1. **Sem Infração:** Se a velocidade do veículo for menor ou igual à permitida.
2. **Infração Média:** Se a velocidade do veículo for superior à permitida, mas o excesso não passar de **20%** do limite da via.
3. **Infração Grave:** Se a velocidade do veículo for superior à permitida em **mais de 20%**.

Exemplos:

- Entrada: Limite 100 km/h | Veículo 115 km/h
- Saída: "Infração Média! (Excesso de até 20%)“

- Entrada: Limite 60 km/h | Veículo 80 km/h
- Saída: "Infração Grave! (Excesso acima de 20%)"

Operadores lógicos

Operador	Significado
&	AND
|	OR
^	XOR (exclusive OR)
||	OR de curto-circuito
&&	AND de curto-circuito
!	NOT

Operadores lógicos de curto-circuito

Java fornece versões especiais de *curto-circuito* de seus operadores lógicos AND e OR que podem ser usadas para produzir código mais eficiente. Para entender o porquê, considere o seguinte. Em uma operação AND, se o primeiro operando for falso, o resultado será falso não importando o valor do segundo operando. Em uma operação OR, se o primeiro operando for verdadeiro, o resultado da operação será verdadeiro não importando o valor do segundo operando. Logo, nesses dois casos, não há necessidade de avaliar o segundo operando. Quando não avaliamos o segundo operando, economizamos tempo e um código mais eficiente é produzido.

O operador AND de curto-circuito é **&&** e o operador OR de curto-circuito é **||**. Seus equivalentes comuns são **&** e **|**. A única diferença entre as versões comum e de curto-circuito é que a versão comum sempre avalia cada operando e a versão de curto-circuito só avalia o segundo operando quando necessário.

```java
public class CurtoCircuito {
    public static void main(String[] args) {
        int num1 = 10;
        int num2 = 2;

        if (num2 > 0 && num1 % num2 == 0) {
            System.out.println(num2 + " é fator de " + num1);
        }

        num2 = 0;
        /*Neste exemplo não ocorrerá erro, pois com o operador
         * de curto-circuito impedirá que a segunda condição,
         * que envole uma divisão, seja verificada.
         */
        if (num2 > 0 && num1 % num2 == 0) {
            System.out.println(num2 + " é fator de " + num1);
        }
        /*Neste exemplo teremos um erro em tempo de execução
         * Pois a segunda condição será verificada independente
         * do resultado da primeira.
         */
        if (num2 > 0 && num1 % num2 == 0) {
            System.out.println(num2 + " é fator de " + num1);
        }
    }
}
```

Ciência da Computação

Fator em Matemática (Divisor)

- Na matemática, um fator é cada um dos números que multiplicamos para chegar a um resultado (produto). Por exemplo, em **2 x 5 = 10**, os números 2 e 5 são fatores de **10**.
- Na programação, dizemos que um número ***x*** é fator de ***y*** se a divisão de ***y*** por ***x*** for exata, ou seja, se o resto da divisão for zero.
- No Java, verificamos isso usando o operador de módulo (%):
  - 10 % 2 == 0 (Verdadeiro: 2 é fator de 10)
  - 10 % 3 == 0 (Falso: 3 não é fator de 10)

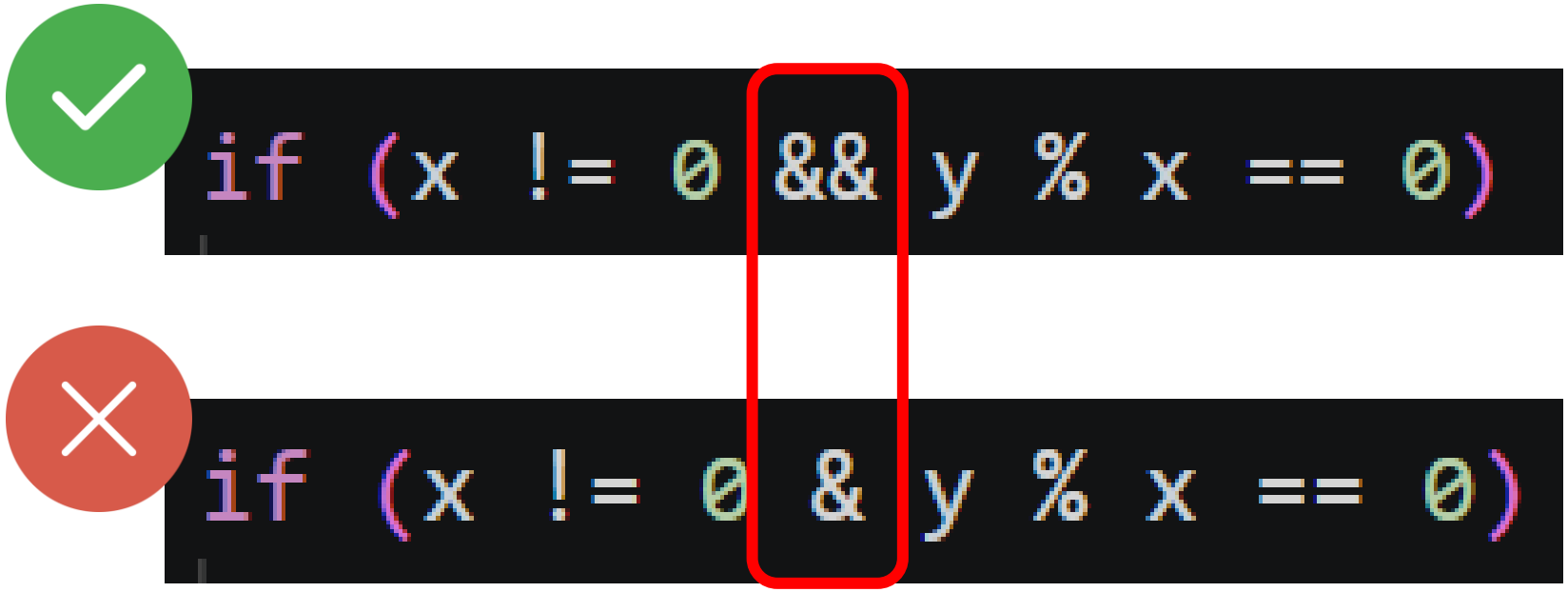
Exemplos e o Perigo do Zero

- Para ser um fator, o número deve ser diferente de zero, pois não existe divisão por zero.
- Se tentarmos fazer **10 % 0** no Java, o programa sofrerá um erro e irá travar.
- Desta forma, antes de tentarmos fazer esse cálculo, devemos confirmar se o segundo valor NÃO é zero.

If (val2 != 0)

Aplicando Operador Lógico de Curto-Circuito (&&)

- Usamos o **operador de curto-circuito** para garantir que o Java só tente calcular o fator se o número for seguro (diferente de zero).



Operador Ternário

- Java também suporta o **operador ternário (? :)**, que permite criar estruturas condicionais de forma mais concisa.

condição **?** se_true **:** se_false;

```
int numero = 5;
String resultado = (numero % 2 == 0) ? "par" : "ímpar";
System.out.println("O número é " + resultado);
```

Instrução **Switch**

- A instrução **switch** fornece uma ramificação com vários caminhos.
- Semelhante às instruções de *ifs* aninhados.
- A **expressão** é um valor ou uma expressão cujo resultado é comparado com os diferentes **case**.
- Os **case** são rótulos que representam os diferentes valores possíveis da expressão.
- O bloco de código associado a cada **case** é executado quando a expressão é igual ao valor do **case**.
- O **break** é usado para sair do bloco de código do **switch** após um **case** ser executado. Isso evita a execução dos **case** seguintes.

```
switch(expressão) {
  case constante1:
    sequência de instruções
    break;
  case constante2:
    sequência de instruções
    break;
  case constante3:
    sequência de instruções
    break;
.
.
.
  default:
    sequência de instruções
}
```

Instrução **Switch**

- As expressões podem ser do tipo **byte, short, int** e **char**. A partir do Java 7 também pode ser do tipo **String**.
- Cada valor de ***case*** deve ser um valor literal constante e de tipo compatível com a ***expressão***.
- Os valores das constantes **não podem repetir** dentro do mesmo switch.
- As instruções da constante **default** são executadas quando nenhuma das outras constantes coincidem com a expressão.
- O **default** é opcional.

```
switch(expressão) {
  case constante1:
    sequência de instruções
    break;
  case constante2:
    sequência de instruções
    break;
  case constante3:
    sequência de instruções
    break;
.
.
.
  default:
    sequência de instruções
}
```

```java
import java.util.Random; //Biblioteca utilizada para gerar números aleatórios

public class ExemploSwitch {
    public static void main(String[] args) {
        Random gerador = new Random();
        int limit = 10;
        int numeroAleatorio = gerador.nextInt(limit);
        switch (numeroAleatorio) {
            case 0:
                System.out.println("O valor gerado é igual a ZERO");
                break;
            case 1:
                System.out.println("O valor gerado é igual a UM");
                break;
            case 2:
                System.out.println("O valor gerado é igual a DOIS");
                break;
            case 3:
                System.out.println("O valor gerado é igual a TRÊS");
                break;
            case 4:
                System.out.println("O valor gerado é igual a QUATRO");
                break;
            default:
                System.out.println("O valor gerado é maior ou igual a CINCO");
                break;
        }
    }
}
```

(JDK 7) Instrução **Switch com Strings**

```java
switch (day) {
    case "Monday":
        System.out.println("Week day");
        break;
    case "Tuesday":
        System.out.println("Week day");
        break;
    case "Wednesday":
        System.out.println("Week day");
        break;
    case "Thursday":
        System.out.println("Week day");
        break;
    case "Friday":
        System.out.println("Week day");
        break;
    case "Saturday":
        System.out.println("Weekend");
        break;
    case "Sunday":
        System.out.println("Weekend");
        break;
    default:
        System.out.println("Unknown");
}
```

Ciência da **Computação**

(JDK 12) Instrução **Switch com Retorno**

```java
public static void main(String[] args) {
    String day = "Tuesday";
    System.out.println(getTipoDia(day));
}
static String getTipoDia(String day) {
    return switch (day) {
        case "Monday":
            yield "Weekday";
        case "Tuesday":
            yield "Weekday";
        case "Wednesday":
            yield "Weekday";
        case "Thursday":
            yield "Weekday";
        case "Friday":
            yield "Weekday";
        case "Saturday":
            yield "Weekend";
        case "Sunday":
            yield "Weekend";
        default:
            yield "Unknown";
    };
}
```

Ciência da **Computação**

(JDK 12) Instrução **Switch com Retorno com arrow**

```java
public static void main(String[] args) {
    String day = "Tuesday";
    System.out.println(getTipoDia(day));
}
static String getTipoDia(String day) {
    return switch (day) {
        case "Monday"→ "Week day";
        case "Tuesday"→ "Week day";
        case "Wednesday"→"Week day";
        case "Thursday"→"Week day";
        case "Friday"→"Week day";
        case "Saturday"→ "Weekend";
        case "Sunday"→ "Weekend";
        default→"Unknown";
    };
}
```


Ciência da Computação

(JDK 12) Instrução **Switch com Múltiplos valores em case**

```java
public static void main(String[] args) {
    String day = "Tuesday";
    System.out.println(getTipoDia(day));
}
static String getTipoDia(String day) {
    return switch (day) {
        case "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" → "Week day";
        case "Saturday", "Sunday" → "Weekend";
        default → "Unknown";
    };
}
```

Agora é com Vocês ...

Atividade Avaliativa:

- Escreva um algoritmo que receba do usuário via teclado (IO.readln) o **nome**, **gênero**, **altura** e **peso**
- Após, faça a classificação conforme a tabela no próximo slide:

- **Observações:**
  - Neste nosso exemplo vamos solicitar que o usuário digite o gênero com as seguintes possibilidades:
    - ‘M’ para Masculino
    - ‘F’ para Feminino
    - ‘N’ para aqueles que não desejarem informar (para esses deverá ser usado o padrão Feminino da tabela)
  - Cálculo IMC = Peso / (Altura²)
  - O método **readln()** da classe **IO** sempre retorna uma String, sendo assim, para receber um **char**, é preciso extrair somente o primeiro caractere com o método **charAt()** da classe String conforme o exemplo abaixo:
    - Char genero = IO.readln(“Digite o gênero”).charAt(0);

IMC

Classificação	Masculino	Feminino
Obesidade Mórbida	Maior ou igual 40	Maior ou igual 39
Obesidade Moderada	30 a 39,9	29 a 38,9
Obesidade Leve	25 a 29,9	24 a 28,9
Normal	20 a 24,9	19 a 23,9
Abaixo do Normal	Menos de 20	Menos de 19

Fica a Dica

Utilizem o recurso Text Block disponível como um recurso padrão desde a versão Java 15

```java
    String textBlock = """
            Aqui temos um exemplo de Bloco de Texto
                --- Você poderá imprimir essa variável 'textBlock'
                --- Será mostrado na tela, da forma como está aqui no código
                --- Inclusive com a Identação
            FICA A DICA GALERA!!!!
            """;
    System.out.println(textBlock);
```

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS

```
Aqui temos um exemplo de Bloco de Texto
    --- Você poderá imprimir essa variável 'textBlock'
    --- Será mostrado na tela, da forma como está aqui no código
    --- Inclusive com a Identação
FICA A DICA GALERA!!!!
```

Limpar a tela

Embora o Java não possua um método nativo para limpar o terminal, existem **diversas maneiras de alcançar esse objetivo**, cada uma com suas vantagens e desvantagens.

Imprimindo caracteres de escape:
Utilize a sequência de escape \033[H\033[2J dentro de uma String e imprima na tela:

IO.println("\033[H\033[2J");

Vantagens:
Portabilidade: Funciona em diferentes sistemas operacionais, pois utiliza caracteres de escape ANSI.
Desvantagens:
Menos intuitivo: A sequência de escape pode parecer complexa para alguns usuários.